



Junkyard Computing: Repurposing Discarded Smartphones to Minimize Carbon

Jennifer Switzer
jfswitze@ucsd.edu
UC San Diego
La Jolla, CA, USA

Gabriel Marcano
gmarcano@ucsd.edu
UC San Diego
La Jolla, CA, USA

Ryan Kastner
kastner@ucsd.edu
UC San Diego
La Jolla, CA, USA

Pat Pannuto
ppannuto@ucsd.edu
UC San Diego
La Jolla, CA, USA

ABSTRACT

1.5 billion smartphones are sold annually, and most are decommissioned less than two years later. Most of these unwanted smartphones are neither discarded nor recycled but languish in junk drawers and storage units. This computational stockpile represents a substantial wasted potential: modern smartphones have increasingly performant and energy-efficient processors, extensive networking capabilities, and reliable built-in power supplies. This project studies the ability to repurpose these unwanted smartphones as “junkyard computers.” Junkyard computers grow global compute capacity by extending device lifetimes, and save carbon by supplanting the manufacture of new devices. We show that the capabilities of even decade-old smartphones are within those demanded by modern cloud microservices, and discuss how to combine phones to perform increasingly complex tasks. We describe how current operation-focused metrics do not capture the actual carbon costs of compute. To address this, we propose Computational Carbon Intensity—a performance metric that balances the continued service of older devices with the superlinear runtime improvements of newer machines. We use this metric to redefine device service lifetime in terms of carbon efficiency. We develop a cloudlet of reused Pixel 3A phones and analyze the carbon benefits of deploying large, end-to-end microservice-based applications on these smartphones. Finally, we describe system architectures and associated challenges to scale to cloudlets with hundreds and thousands of smartphones.

CCS CONCEPTS

• **Computer systems organization** → **Cloud computing**; *Embedded and cyber-physical systems*.

KEYWORDS

sustainability, cloud computing, life cycle assessment

ACM Reference Format:

Jennifer Switzer, Gabriel Marcano, Ryan Kastner, and Pat Pannuto. 2023. Junkyard Computing: Repurposing Discarded Smartphones to Minimize Carbon. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '23)*, March 25–29, 2023, Vancouver, BC, Canada. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3575693.3575710>



This work is licensed under a Creative Commons Attribution 4.0 International License.

ASPLOS '23, March 25–29, 2023, Vancouver, BC, Canada

© 2023 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9916-6/23/03.

<https://doi.org/10.1145/3575693.3575710>

1 INTRODUCTION

Manufacturing electronic devices is an energy-intensive process. For devices with lower utilization, such as consumer-class electronics or over-provisioned servers, manufacturing dominates the lifetime carbon footprint [17]. This is especially true in devices with short use cycles. In the United States alone, 150 million smartphones are discarded each year, amounting to one phone discarded per person every two years [8]. As a result, manufacturing accounts for 70 – 85% of the lifetime carbon footprint of a smartphone [17, 31].

Consumer electronics are also becoming increasingly powerful—the performance of recent smartphones rivals or exceeds that of an Intel Core-i3 processor (Figure 1). Yet, phones are often discarded despite being completely (or partially) operational. Otherwise functional electronic devices are retired prematurely due to technical, style, or planned obsolescence [33]. Compare this to other high-priced devices with entire ecosystems of functional obsolescence, e.g., cars, which are resold until they are “driven into the ground.”

Of course, the performance of computing devices improves much faster than that of a modern automobile. What should the lifetime target be for compute? Should we also run every electronic device “into the ground?” If so, which applications should run on these older, less-capable devices? Can we aggregate devices to perform equivalent larger-scale computing? And perhaps most importantly, how do we assess whether it is worth it: when are there carbon savings to be had by scavenging devices from the “junkyard?”

To answer these questions, we introduce a new carbon-aware performance metric. *Computational Carbon Intensity (CCI)* measures the lifetime carbon impact of a device versus the lifetime useful compute it performs. CCI quantifies the value of extending the service lifetime of computational devices.

We apply CCI to old servers, old laptops, and old smartphones. While each device type shows potential as carbon-saving hardware, we find that used smartphones (repurposed as general-purpose compute nodes) offer the best potential for carbon impact.

Smartphones are an attractive target for repurposing for several reasons. First, there is a remarkably large number of them. Between 60-70% of smartphones are neither thrown out nor recycled [6, 38]. If even 10% of the devices decommissioned in the last five years were available for repurposing, we would have 75 million new compute nodes. Second, a smartphone comes with a wide array of valuable components: increasingly powerful processors, a robust uninterruptible power supply (batteries), and a diverse array of networking hardware, which includes local area connectivity (WiFi and Bluetooth) and long-range, high-performance uplink (cellular modems).

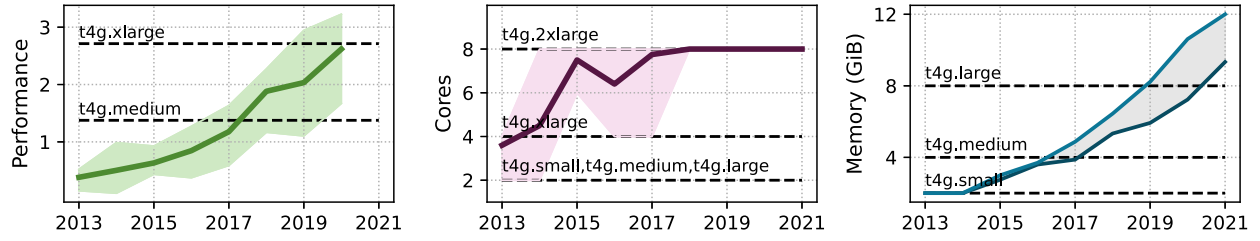


Figure 1: The capabilities of recent smartphones meet or exceed that provided for modern microservices. The three plots show the performance (according to the GeekBench score[21]), number of cores, and memory for the five most popular Android phones released each year since 2013. A GeekBench score of 1 is equivalent to an Intel Core i3 processor. Solid lines indicate the mean. The shading shows the minimum and maximum ranges. The memory plot has two lines corresponding to the minimum- and maximum-memory configurations available. The horizontal dotted lines show the capabilities of different Amazon EC2 T4g instance sizes as of August 2021, plotted for context.

Can a smartphone meet the performance provided by purpose-built cloud servers? We argue that it can. Figure 1 compares the performance, the number of cores, and memory in legacy smartphones to AWS T4g instances, which use Arm-based AWS Graviton2 processors. T4g instances target burstable general-purpose workloads like microservices, e-commerce platforms, small databases, development environments, and virtual desktops [2]. The analysis indicates that even a several-year-old smartphone could be valuable for this class of modern cloud service.

We put this idea to the test. We benchmark discarded smartphones, some of which are now a decade old. We then build out a cloudlet of Pixel 3A smartphones, purchased on eBay for \$65 USD. We measure the performance of the cloudlet using DeathStarBench, a benchmarking suite for cloud microservices [19].

The main contributions are:

- (1) Defining CCI, a new metric to quantify the lifetime carbon impact of a device and our ability to amortize that impact (Section 3)
- (2) Comparing the carbon and performance trade-offs of reusing old servers, laptops, and smartphones to identify the phone as a first-priority platform to explore for reuse (Section 3)
- (3) Performing a design space exploration of how to architect legacy phones into a ‘junkyard computer’ (Section 4)
- (4) Building a real-world prototype cloudlet constructed from reused devices and benchmarking its performance using modern microservice applications from the DeathStarBench suite (Section 6).

2 RELATED WORK

We are not the first to highlight the computational capability of wimpy cores [3], nor the first to suggest that phone-class hardware could be used [23, 43], or re-used [27, 50], in cloud computing contexts. What differentiates this work is centering the design and evaluation on the amortized lifetime carbon impact and empirical testing on recovered hardware.

2.1 Recycling is not the Answer

E-waste recycling of consumer electronics recovers less than 50% of their materials [52] and has a negative local impact on people

and the environment [7, 28]. The inclusion of precious metals in ICs provides an incentive for recycling, but the presence of toxic chemicals means that the recycling process is energy-intensive and hazardous [37]. Wealthy countries in North America and the EU exacerbate global health disparities by shipping large amounts of E-waste to developing nations such as China and India. Just three towns in China are estimated to process 11.5% of the world’s E-waste [9]. Extensive studies of one of these towns revealed dangerous levels of pollutants in the surrounding environment [39] and elevated levels of heavy metal poisoning among residents.

2.2 Computational Efficiency Metrics

While efficiency metrics are useful for many applications, they do not capture the environmental and human cost of compute. Greater energy efficiency does not always imply a lower carbon footprint, especially when efficiency is achieved via fast device turnover.

Power Usage Effectiveness (PUE) is a standard metric for datacenter efficiency that expresses the ratio of the total energy consumption of the datacenter to the energy consumption of ICT devices alone, with 1.0 being an ideal value. While PUE captures the operational efficiency of the datacenter’s computational devices and cooling system, it does not reflect the tremendous energy that went into manufacturing the facility or the equipment housed within.

To understand why this is important, consider two datacenters, A and B. For the same computational output, datacenter B consumes 10% more energy than A, i.e., $PUE_{(B)} = 1.1 \times PUE_{(A)}$. However, datacenter A achieves this efficiency by upgrading its servers at a rate that is 2× faster than datacenter B; i.e., datacenter A’s ICT manufacturing costs are 2× that of datacenter B. Let us assume that manufacturing is responsible for 20% of the carbon emissions associated with both datacenters and that operational energy accounts for the other 80%—a reasonable assumption for a server-class machine [10]. Then, the carbon footprint of datacenter B compared to that of datacenter A is:

$$\begin{aligned} CO_2e_{(B)} &= 0.8 * (1.1 \times CO_2e_{(A)}) + 0.2 * (0.5 \times CO_2e_{(A)}) \\ CO_2e_{(B)} &= 0.98CO_2e_{(A)} \end{aligned}$$

Where CO_2e is the Global Warming Potential (GWP), or carbon footprint, in units of CO_2 -equivalent weight. Despite having a higher

PUE, datacenter B has a slightly lower carbon footprint because it uses its servers longer.

Total Cost of Ownership (TCO) reflects the total dollar cost of equipment and operations over a period of time. It reflects how well initial purchases costs are amortized, and captures runtime efficiency via operational overhead. However, these economic costs are not always aligned with carbon costs.

2.3 Characterizing the Carbon Footprint of Computation

Several recent publications bring attention to the growing importance of carbon metrics in computing. Patterson et al. argue that researchers should consider the carbon emissions incurred by machine learning training [45]. Their formula for the carbon emissions of training is similar to our definition of compute carbon (C_C , introduced in the next section) with the exception of scope: we calculate operational carbon for the entire lifetime of a system as opposed to a single model training event. In both cases, the energy source is important to the overall carbon footprint. Patterson et al. do not address embodied carbon.

Gupta et al. discuss the importance of accounting for embodied carbon [25]. Like us, they note that a significant fraction of smartphone carbon emissions come from manufacturing. They further note that recent optimizations have focused on maximizing performance rather than considering carbon footprint. Like us, they argue for longer device lifetimes: they state that amortizing the carbon footprint of mobile devices requires continuously operating them for at least three years. In follow-on work, Gupta et al. develop carbon-conscious metrics to enable architectural-level modeling of carbon consumption and describe the carbon benefits of repurposing smartphones as an SSD storage system [24].

Life-cycle assessment (LCA) seeks to characterize the lifetime environmental impact of a product across multiple metrics, including carbon. LCAs have been performed for smartphones, laptops, and servers [15, 17]. Manufacturing accounts for 70 – 90% of the lifetime carbon footprint for phones and laptops. For high-performance computing devices, manufacturing accounts for 20 – 50% of the lifetime carbon cost [15, 17, 31]. LCAs measure the total carbon footprint of a device across its lifetime; our approach (CCI) is different in that we consider the amortized carbon footprint of each unit of computational work. We use vendor-provided LCA numbers to parameterize our CCI calculation.

Raghavan and Ma characterize the *embodied energy* of the internet [46]. Subsequent work argues for the importance of including embodied energy in discussions of sustainable computing more generally [44]. With this work, we attempt to take a step towards fulfilling this mandate by providing architects and systems engineers the tools to express embodied energy quantitatively.

2.4 Repurposing Consumer Electronics for HPC

Rajovic et al. propose using mobile SoCs for high-performance computing (HPC) [47]. Their analysis finds that mobile SoCs are sufficiently performant for many applications and are more energy efficient than traditional HPC chips. Their follow-on work builds a mobile SoC-based cluster [48]. Their implementation uses new and isolated mobile SoCs and is not focused on reuse.

Shahrad and Wentzlaff propose a server built from decommissioned mobile phones [50]. While the work considers E-waste reduction as motivation, it does not look into quantifying the carbon impact. They present a design proposal but do not include an implementation or empirical evaluation.

Büsching et al. empirically test six Android phones over WiFi to evaluate cluster performance on LINPACK [11]. They do not consider other workloads, architectures, or management costs.

2.5 Repurposing Smartphones as Sensors

An orthogonal line of work is using smartphones in IoT and sensor networks [36, 53]. Mobile phone operating systems are optimized for interactive use, and when human interaction is completely removed, a very long tail of deployment-ending issues crops up. This experience led us to replace the standard mobile phone operating system with a more traditional technology stack for our smartphone-based system.

3 QUANTIFYING CARBON

The first step in repurposing devices is assessing their performance and carbon consumption tradeoffs versus a new system. Can we quantify the carbon benefit of repurposing devices? How much performance are we giving up by not using the latest technology? Which device provides the best tradeoff between performance and environmental impacts?

We study the performance and carbon benefits of building a server using different legacy devices. Our baseline is a PowerEdge R740—a modern server for which Dell has published a full LCA [10]. We aim to replicate the R740 functionality with repurposed devices. We compare four building blocks: a 15-year-old server (HP ProLiant DL380 G6), an 8-year-old laptop (Lenovo Thinkpad X1 Carbon G3), a decade-old smartphone (Nexus 4), and a 3-year-old smartphone (Pixel 3A).

3.1 Workload

We assume that the devices operate at the light-medium operating regime specified in Dell’s PowerEdge R740 LCA [10]. This operating regime has the following load profile:

- 100% load mode: 10% of the time.
- 50% load mode: 35% of the time.
- 10% load mode: 30% of the time.
- Idle mode: 25% of the time.

We further consider what supporting datacenter infrastructure must be added to each configuration to provide the following:

- 1 Gbps networking capacity.
- 30 minutes backup power.
- Sufficient cooling to operate in 25 C ambient temperature.

3.2 Performance

Table 1 shows the performance of the five different devices across four applications from the Geekbench suite.¹ The PowerEdge Server has the highest performance, followed by the old server, old laptop,

¹We generally eschew performance benchmarks such as SPEC due to their unavailability and inaccessibility for mobile platforms. To date, Geekbench is the most commonly used measure in studies that focus on a broader array of platforms, particularly those which include mobile platforms [25, 50].

Table 1: The GeekBench performance on four benchmarks across the five devices. ‘Single’ represents the single-core performance, and ‘multi’ represents the multi-core performance. We consider the latter to be representation of the total computational power of the device. N is the number of devices that would be required to build a system with approximately the same computational power as a single PowerEdge R740 (baseline). This is calculated by dividing the PowerEdge multicore throughput by that of the reused device.

		SGEMM (Gflops)			PDF Render (Mpixels/sec)			Dijkstra (MTE/sec)			Memory Copy (GB/sec)		
		Single	Multi	N	Single	Multi	N	Single	Multi	N	Single	Multi	N
PowerEdge R740 Server	2017	77.2	2,070	1	109.1	3,140	1	3.58	80.2	1	6.33	19.5	1
HP ProLiant DL380 G6	2007	14.2	104.2	20	74.2	528.4	6	2.43	16.9	5	6.52	11.3	2
Lenovo Thinkpad X1 Carbon G3	2015	72.1	123.7	17	123.2	225.1	14	3.08	7.45	11	11.0	13.1	2
Pixel 3A Smartphone	2019	8.84	39.0	54	38.9	147.0	22	1.08	4.44	19	4.00	5.45	6
Nexus 4 Smartphone	2012	1.95	8.12	256	14.1	40.8	77	0.654	2.21	37	2.35	3.22	7

and old smartphones. N provides the performance ratio between the PowerEdge Server and other devices. That value changes depending on the application. For example, there is a 256 $\times$ difference between the PowerEdge Server and the Nexus 4 for SGEMM, while the relative performance difference is just 7 $\times$ for Memory Copy.

3.3 Energy Consumption

Device power draw plays a prominent role in the overall carbon footprint. We perform a CPU stress test on all devices to characterize their power draw in different regimes. We turn off WiFi and Bluetooth on the phone and laptop for the stress test. We turn off the phone screens and turn the laptop’s brightness as low as possible. We run the Linux `yes` command n times, where n is the number of cores the device has and use `cpulimit` to restrict the CPU usage of each `yes` process. The phone’s power draw is measured via a USB power meter, which sits between the device and the power source. The laptop measurements use a plug-through power meter. We query the HPE iLO interface for the ProLiant servers.

Table 2 presents a summary of the results. The servers have the highest power draw, followed by the laptop and smartphones.

Table 2: Power (Watts) given CPU usage. Subscripts indicate CPU usage percentage. P_{avg} calculated for light-medium workload.

	P_{100}	P_{50}	P_{10}	P_{idle}	P_{avg}
PowerEdge	510	369	261	201	308.7
ProLiant	280	213	181	169	199.1
Thinkpad	24	16.2	8.5	3.4	11.47
Pixel 3A	2.5	1.9	1.4	0.8	1.54
Nexus 4	3.6	2.7	1	0.7	1.78

3.4 Carbon Cost

To quantify the environmental cost of proposed solutions beyond operational energy, we need new metrics that properly capture the carbon footprint over the system’s lifetime. An *ideal carbon metric*:

- (1) Rewards operational energy efficiency.
- (2) Rewards manufacturing efficiency.
- (3) Rewards the reuse of already-manufactured devices.
- (4) Reflects computational work achieved per unit carbon.

Existing efficiency metrics, e.g., PUE, fulfill point #1 but fail to capture points #2-4. Carbon footprint—a measure of the greenhouse

gas emissions of a device over a standard lifetime, reported by companies like Apple [4, 5] and Dell [10, 15]—does not satisfy point #4. Thus, we must define a metric that captures all four points.

Computational Carbon Intensity (CCI) is the CO₂-equivalent released per unit of computation work. We calculate this metric across the entire lifespan of our devices to get their amortized carbon intensity. The general formula is as follows:

$$CCI = \frac{\sum_{lifetime} CO_2e}{\sum_{lifetime} ops} \quad (1)$$

The numerator can be further broken up into the carbon associated with manufacturing, compute, and networking:

$$CCI = \frac{C_M + C_C + C_N}{\sum_{lifetime} ops} \quad (2)$$

C_M is the carbon associated with the manufacture of the device (often called the “embodied carbon”). It is a single upfront cost at the beginning of the cluster’s lifetime. We source our values for C_M from the life cycle assessment literature.

C_C is the carbon associated with compute. Equivalently, it is the total carbon footprint of the energy required to perform the computations over the lifetime of the device, calculated as:

$$C_C = \sum_{lifetime} CI_{grid} * E \quad (3)$$

where CI_{grid} is the carbon intensity of the grid, in units of $\frac{CO_2e}{joule}$, and E is the cluster’s energy consumption, in joules. At the light-medium workload, this works out to:

$$C_C = CI_{grid} * \sum_{lifetime} 0.10 * P_{100\%} + 0.35 * P_{50\%} + 0.30 * P_{10\%} + 0.25 * P_{idle} \quad (4)$$

C_N is the carbon associated with networking:

$$C_N = \sum_{lifetime} CI_{grid} * f_{net} * EI_{net} \quad (5)$$

where f_{net} is the rate at which data is sent and received, in $\frac{bytes}{second}$, and EI_{net} is the energy intensity of networking, in $\frac{joule}{byte}$. CCI sums these three carbon components and divides by the total number of operations computed in the device’s lifetime.

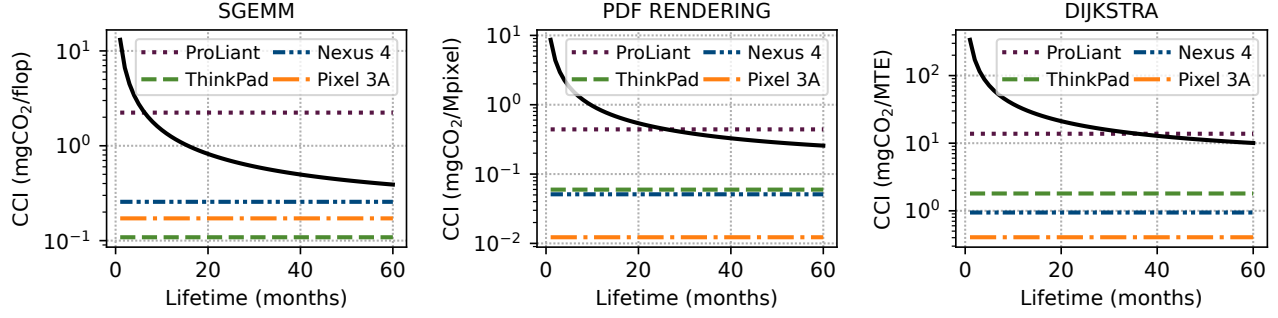


Figure 2: Single-device CCI trends for three benchmarks. Lower CCI values are better. This analysis assumes no peripherals are needed and a California energy mix. The chosen benchmark affects the relative performance of each device. In all cases, the CCI of the reused smartphones and laptop is lower than that of a new server. The reused server does not perform as well due to its poor performance/Watt.

CCI satisfies point #1 as it accounts for operational energy consumption per instruction in the numerator (C_C). It satisfies point #2 via the inclusion of C_M . The total carbon cost is amortized per instruction; thus, point #4 is satisfied. To satisfy point #3, we stipulate that when reusing a device, the carbon cost of manufacturing is considered already paid, i.e., $C_M = 0$.

CCI in practice. With this definition, we can do an initial, per-device CCI calculation for the servers, laptop, and phones.

C_C is calculated per Equation (4) using the P_{avg} from Table 2 and the throughput from Table 1. We do not include the networking term C_N for this single device analysis. As the reused devices are already manufactured, we set their $C_M = 0$.

Operations per second (ops) varies depending on the computational work done. For example, the GeekBench SGEMM benchmark is measured in floating point operations, PDF Rendering uses pixels, Dijkstra computes pairs, Memory Copy is GB, etc. [20]. CCI depends on the type of operation; thus, its units can change depending on the benchmark under consideration. Operations are calculated as follows: We scale the throughput achieved in the micro-benchmark under consideration by the CPU usage to get operations per second, then multiply by the lifetime to get the total operations computed.

We assume that throughput scales linearly with CPU usage, e.g. $ops_{50\%} = 0.5 * ops_{100\%}$. This simplification is necessary when extrapolating from microbenchmarks; when benchmarking a full system (Section 6), we no longer make this assumption. Under the light-medium workload described previously, the average operations per second (ops) is calculated as:

$$ops_{avg} = 0.10 * ops_{100\%} + 0.35 * ops_{50\%} + 0.30 * ops_{10\%} \quad (6)$$

Figure 2 shows the CCI over the lifetime of the five devices. Lower CCI values are better. Only the PowerEdge server incurs manufacturing carbon cost C_M ; repurposed devices have $C_M = 0$. Generally speaking, the phones have the best (lowest) CCI except in the SGEMM benchmark. Here, the specialized computational hardware available on the laptop outweighs its higher power draw.

Assumptions and limitations. It is difficult to account for every possible impact of a computing system. For instance, CCI, as it

stands, does not account for the carbon footprint of human labor and transportation beyond initial manufacture.

For our analysis, we assume that any device being repurposed is essentially ‘free’ in terms of manufacturing carbon, i.e., $C_M = 0$.² An alternate analysis considers the initial manufacturing cost of the repurposed device, amortized by the work done in its first life. This alternate CCI formula is:

$$CCI = \frac{C_M + C_C(1^{st}) + C_N(1^{st}) + C_C(2^{nd}) + C_N(2^{nd})}{\sum_{1^{st} \text{ lifetime}} ops(1^{st}) + \sum_{2^{nd} \text{ lifetime}} ops(2^{nd})} \quad (7)$$

Where $C_C(1^{st})$ is the operational carbon footprint of the first life, $C_C(2^{nd})$ is the operational carbon footprint of the second, etc. While this alternate form might be helpful for certain analyses, we find that it is difficult to reason about the operational carbon footprint and operations computed in a device’s ‘first life.’ In our case, we are working with used devices from eBay and have no visibility into their first lives.

Another limitation of CCI is that it focuses on computing and does not capture other types of reuse, e.g., it does not account for whether or not the camera, microphone, and networking capabilities are being reused. Consumer devices contain many subcomponents supporting different compute requirements: CPUs, GPUs, custom hardware accelerators (audio, encryption, AI), networking hardware, power supplies, and batteries. Ideally, every component is reused, but in reality, not all are required. For example, devices in a server role may not need a display or audio components.

Reuse Factor is a metric to account for the usage of the subcomponents. It weighs each component by its carbon footprint to estimate the embodied carbon of the components of the device that are being reused. It is calculated as:

$$RF = \frac{\sum_{reused} C_{M(i)}}{C_M} \quad (8)$$

Table 3 gives our best attempt at capturing the component-wise carbon footprint of a smartphone. Coarse-grained ratios are sourced from [17], which reports a 77% contribution from ICs, 10% from the

²Although as we will see below, when building more complex systems the carbon cost of added peripherals must be considered.

Table 3: Working estimates for the fractional carbon intensity of various Nexus 4 subcomponents.

Category	Contribution	Includes
Compute	25% 12.5 kgCO ₂	2GB RAM (Samsung K3PE0E00A), Snapdragon S4 Pro
Network	15% 7.5 kgCO ₂	LG and WiFi chips: Qualcomm MDM9215M, Murata SS2908001, Qualcomm WTR1605L, Broadcom 20793S
Battery	15% 7.5 kgCO ₂	Battery, power management (Qualcomm PM8921, Avago ACPM-7251)
Display	10% 5 kgCO ₂	Screen
Storage	10% 4 kgCO ₂	8GB flash
Sensors	5% 3 kgCO ₂	Accelerometer, camera, audio codec
Other	20% 10 kgCO ₂	PCB, Chassis, packaging, other ICs

display, 3% from the battery, and the rest from other components, including the PCB. The contribution from ICs was then further broken down using images from a Nexus 4 teardown and assuming that the relative contribution of each chip scales according to size.

Better component-wise embodied carbon estimates are necessary to create more accurate models. The ACT is a tool for modeling architectural carbon footprint [24]. Currently, ACT only has estimates for ICs, which align with our estimates here, but does not consider all elements of the phone (e.g., battery, display, PCB, and chassis, which contribute $\sim 1/3$). Thus, ACT is complementary to our work, and we are excited by the potential of its component models to improve our more holistic models.

The reuse factor should be treated as a proxy for the extent to which a device has been reused. To see how the reuse factor is used, consider a cloudlet example: A smartphone is repurposed as a compute node. It is network-connected and is always in use. Code and results may use storage on the device and its battery as a UPS. In this scenario, the compute capabilities, networking, battery, and storage are all reused, while the display and sensors are not. This yields a reuse factor of 0.85.

4 DESIGNING THE JUNKYARD

We showed that repurposed smartphones have the potential to outperform a traditional server in terms of carbon per op. This section discusses how to transform discarded consumer smartphones into general-purpose compute nodes. We aim to answer the question: What does it take to make a server out of smartphones?

4.1 Cooling

Datacenter cooling can account for as much as 40% of operational energy consumption [26]. While we envision cloudlets rather than warehouses as a more realistic outcome for repurposed phones, we still anticipate packing heat-generating compute into limited space.

Phones are designed not to burn your hand when you use them, which means they have a strict upper limit on their thermal dissipation. They begin throttling around 40–50°C [32]. Thus, a well-behaved device should never overheat due to its operation. At around 60–70°C, the device will shut itself off to protect the user [35].

While collecting many smartphones in an enclosed space can create a high ambient temperature, we postulate that the individual

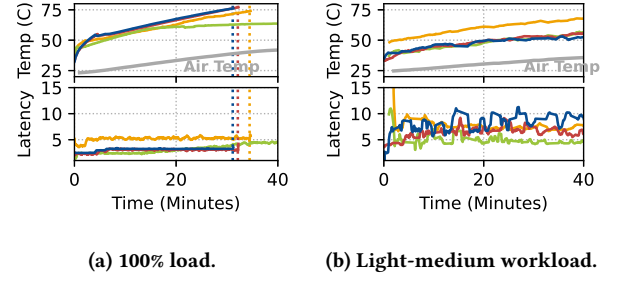


Figure 3: Results of a thermal stress test. The top graph shows the temperature internal to each phone (colored lines) and the external air temperature (grey line). The bottom graph shows the test job latency. For both graphs, the green line is the Nexus 5; other colored lines are the Nexus 4s. The horizontal dotted lines at the ends of the Nexus 4 curves represent the device shutting itself off.

thermal throttling of each device has the aggregate effect of limiting the air temperature reached.

Experimental Set-Up. Four Nexus 4s and one Nexus 5 are placed in a sealed $5 \times 15 \times 10.5$ inch Styrofoam box. We run two scenarios: a CPU stress test, with a 90%+ CPU job running on all four cores, and a simulated light-medium workload. We log job throughput, device internal temperature, and air temperature in the box.

Results. Figure 3 summarizes performance and temperature results. We also calculate thermal power for both scenarios:³

$$P_T = \frac{c_{p(air)} * m_{air} * \Delta T_{air}}{\Delta time} + \sum_{phones} \frac{c_{p(Si)} * m_{ph.} * \Delta T_{ph.}}{\Delta time} \quad (9)$$

We calculate the thermal power prior to the shutdown of any devices. Total thermal power for the 100% load scenario is ~ 13 W (2.6 W/device), and ~ 6 W (1.2 W/device) for the light-medium scenario.

Some additional observations: a) The phones shut off at an internal temperature of 75–80°C. For most devices, this happened at an air temperature of 40°C. b) The Nexus 5 did not overheat in either scenario. c) As the temperature increased, performance decreased. d) Even under a 90%+ (CPU) load, the phones exhibited a thermal power (2.6 W) well below their 5 W thermal design point (TDP).

At the cloudlet scale, with 256 Nexus 4s running at 100%, we expect up to 666 W of thermal power. This is within the cooling abilities of two COTS server fans rated for 500 W, which would add 4 W of power per fan [16]. A rough estimate based on weight [40], and assuming a world energy mix during production, yields an embodied carbon of 9.3 kg per fan. We consider this added embodied carbon when estimating the carbon efficiency of cloudlet-scale smartphone clusters in Section 5.2.

Scaling Further. Aggressive thermal management reduces the cooling demand of a hypothetical smartphone datacenter but also limits its performance potential. A phone’s modest thermal dissipation capacity means that its effective sustained TDP is much lower than its transient peak TDP. Thus, phones are well-suited to bursty

³We assume the phones can be modeled as a block of silicon and that the temperature of the phones and the air was uniform throughout the medium.

workloads, but a poorer fit for more sustained compute without an aggressive adaptation of their thermal management systems.

4.2 Networking

There are many ways to network the phone cluster. We consider two general use cases: (1) the in-situ, edge compute case, where the cluster operates independently with access only to the cellular network and (2) the existing-infrastructure case, where the cluster has access to a pre-existing WiFi or wired network. The first case reflects applications in remote environments. The second might occur if the cluster was housed within an office building or datacenter.

Na et al. found that networking a co-located smartphone cluster over WiFi is not feasible; interference made wireless networking intractable beyond 30 devices [41]. Thus, in a datacenter-scale scenario, we expect that wired clusters of smartphones connected to network switches would be employed.

For smaller clusters operating away from external infrastructure, we propose a tree topology, with phones organized into sets of five, each with one hotspotted device. The hotspotted device communicates with the outside world via LTE and other devices via its WiFi network. WiFi is the more bandwidth limiting, e.g., the Nexus 4/5 smartphones have 150 Mbit/s for both the uplink and downlink [12]. The tree topology leads to a capacity of 18.5 Mbit/s downlink and uplink per device. While wired networking is best as the cluster scales, for smaller clusters (i.e., our ten-device cloudlet described in Section 6), we have found a wireless network to be sufficient.

4.3 Batteries & Smart Charging

A critical advantage of smartphones is their built-in batteries. By charging during periods of higher renewable (“greener”) energy production, we can reduce the operational carbon intensity of the cluster. We call this *smart charging*.

Smart charging opportunistically charges the devices whenever the grid-level carbon intensity falls below a tunable threshold. This threshold is based on historic grid characteristics and the charged device. We devised a heuristic for the California grid, but other regimes might require a different strategy. The device is charged if the battery level drops below 25%, regardless of grid conditions.

We evaluate this algorithm on publicly available data from the California Independent System Operator [29]. In California, the carbon intensity of the grid tends to be anti-correlated with solar production trends (Figure 4a). Since this pattern is relatively uniform across days, we set the threshold as the Pth percentile of the previous day’s instantaneous carbon intensities, where P is the percent of time the device will need to spend charging.

In California, the smart charging algorithm schedules charging when solar production is highest (middle of the day). The carbon savings vary depending on the device’s battery capacity and charging rate. The Pixel 3A, with a 3 Ah battery and 18 W charging rate, sees a median carbon reduction of 7.22% for the time period studied, while the ThinkPad X1 Carbon Gen3 laptop sees a median 4.03% reduction during the same time period. The laptop sees smaller savings than the Nexus phone because its much higher power consumption (11.4 W versus 1.54 W) offsets its larger battery capacity.

Batteries also provide a convenient source of backup power. For a Pixel 3A operating on a light-medium workload, a 25% charge will

last just under 2 hours. The 25% minimum can be increased to provide a higher margin. Conversely, the minimum charge threshold can be decreased to prioritize carbon savings over backup power.

There is one complicating factor, however: Battery lifetimes. Smartphone batteries become unusable after about 2,500 cycles [18]. Consider a Pixel 3A with a light-medium usage pattern. The mean power draw of the device would be 1.54 W, giving a daily energy consumption of 133 kJ. The 3 Ah (45 kJ) battery included in the Pixel would require three full daily charges. After 833 days or 2.3 years, the battery would be unusable and have to be replaced.

The Pixel 3A’s battery has an embodied carbon of 2.00 kgCO₂e and a projected lifespan of 2.3 years. The Nexus 4’s 2.1 Ah battery and 1.8 W average power yields an embodied carbon of 1.11 kgCO₂e and a projected lifespan of 1.23 years. For any reused smartphone, the embodied carbon cost of replacing the battery C_M is:

$$C_M = C_{M(BATTERY)} * \left[\frac{lifetime}{battery\ lifetime} \right] \quad (10)$$

This replacement is also costly in terms of human labor. We have replaced a Nexus 4 battery and found it takes about 10 minutes. For a server-equivalent cluster of 54 (Pixel 3As) or 270 (Nexus 4) phones, this would imply 9 (Pixel 3A cluster) - 45 (Nexus 4 cluster) hours of labor every 1.2-2.3 years. We believe this is reasonable; however, this upkeep may become prohibitive for larger systems. Removing and bypassing the battery completely might be worthwhile despite the loss of smart charging and using the battery as a UPS.

5 CHARACTERIZING CARBON

We consider the design factors from the previous section to evaluate the carbon savings of junkyard smartphone servers.

5.1 Sourcing Energy

First, we consider the energy source’s effect on the carbon intensity of new and reused devices. The carbon emissions associated with compute, C_C , are equivalent to the carbon footprint of the wallplug energy used to power the device. This can be expressed as:

$$C_C = C_{I_{grid}} * \sum_{lifetime} P_{avg} \quad (11)$$

The carbon intensity of the grid ($C_{I_{grid}}$) is a measure of the amount of CO₂e released per kWh of energy provided and varies depending on the source of that energy. For instance, the carbon intensity of solar is 48 gCO₂-e/kWh and is 602 gCO₂-e/kWh for gas. In California, the mean carbon intensity of grid power is 257 gCO₂-e/kWh. Figure 6 examines how variations in local energy supply affect CCI. Unsurprisingly, more renewable energy means a lower carbon footprint. We can get additional gains with smart-charging.

Figure 6 considers three power regimes. The first is an energy mix that models the California grid. The second is a theoretical regime in which solar energy is always available. While this is not realistic today, hyperscalers have announced plans for 24/7 carbon-free energy within the next decade [22].

The third is a theoretical, 100% carbon-free energy source. Note that such an energy source does not currently exist. Even solar, wind, and hydro have carbon costs associated with their generation, which is reflected in their carbon intensity. However, it provides a

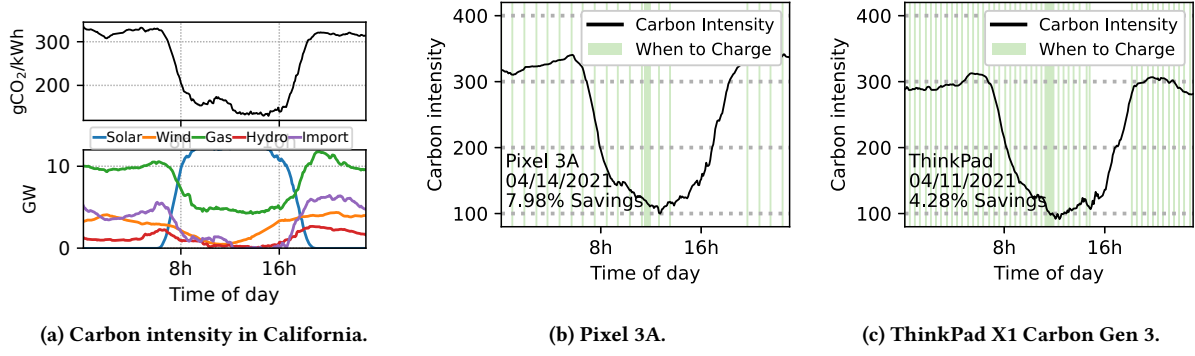


Figure 4: Smart charging saves carbon. Results are shown for a representative day during the month of April 2021. The black curve indicates the carbon intensity of the California grid throughout the day, and the green shading gives the charging periods suggested by the smart charging algorithm. On this particular day, the Pixel 3A's carbon emissions are reduced by 7.98%, and the ThinkPad's carbon emissions are reduced by 4.28%. Over the course of the month as a whole, the Pixel 3A sees median carbon savings of 7.22%, with a standard deviation of 5.93%. The ThinkPad sees median carbon savings of 4.03%, with a standard deviation of 2.2%

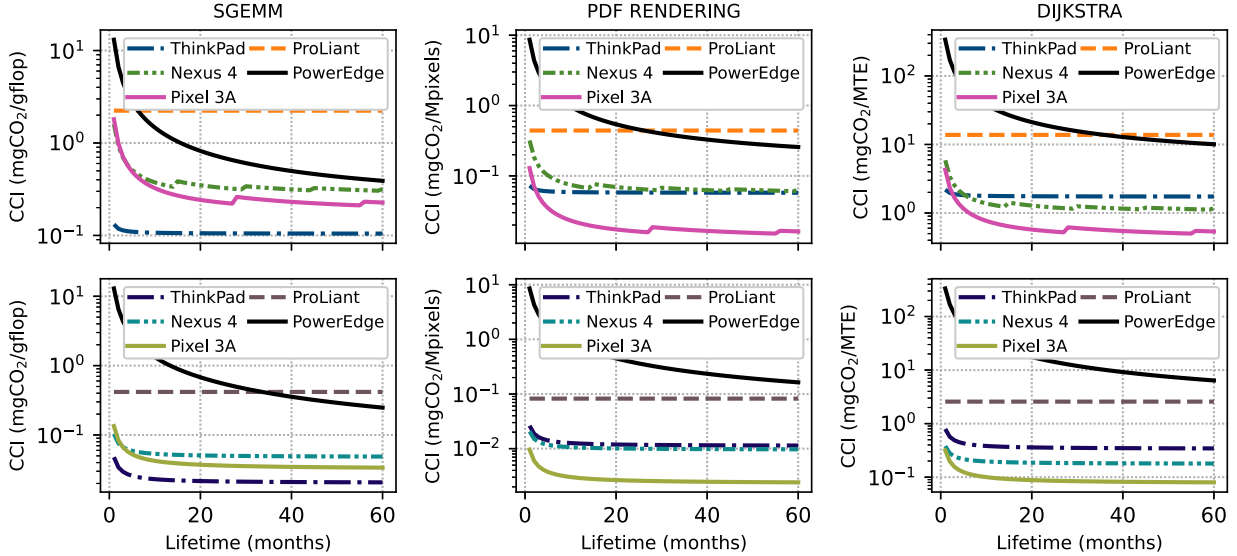


Figure 5: Cluster-level CCI for the smartphone clusters vs. an equivalent number of new PowerEdge servers, old ProLiant servers, and old ThinkPad laptops. The six graphs have three benchmarks with two power regimes. The first row shows CCI for a California energy mix with smart charging for the laptops and smartphones. Smart charging requires a periodic replacement of the battery. The second row assumes access to solar power 100% of the time, obviating the need for charging and eliminating the batteries.

theoretical lower bound. In this case, C_C is zero, and as Figure 6 shows (green lines), C_M becomes the dominant factor for CCI.

5.2 Cloudlet-scale Computing

We now compare the carbon intensities of each previously introduced comparison point at the cloudlet scale. That is, we consider the number of devices and the peripherals necessary to make a cluster of smartphones, laptops, or old servers that meet the performance of a PowerEdge R740 server. We also consider the effect of networking (reflected by the C_N term). Our cloudlets are:

(1) A single PowerEdge R740.

- (2) 17 Lenovo ThinkPad Gen 3s, with 17 smartplugs added to achieve smart charging savings of 4%.
- (3) 20 ProLiant.
- (4) 54 Pixel 3A phones, with 20% designated as networking and management nodes. 54 smartplugs are added to achieve smart charging savings of 7%, and one server fan is rated for a 500 W TDP.
- (5) 256 Nexus 4 phones, with 20% designated as networking and management nodes. 256 smartplugs are added to achieve smart-charging savings of 7% and two server fans rated for a 500 W TDP.

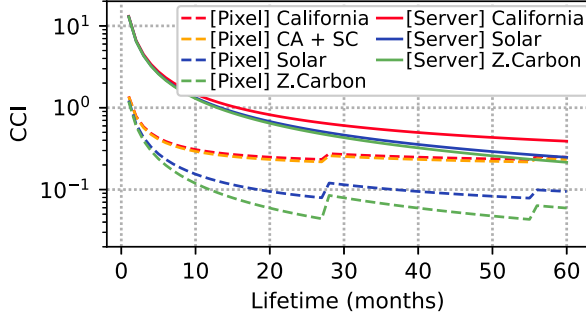


Figure 6: The energy mix has a significant impact on CCI. The curves show the CCI for the PowerEdge server and the Pixel 3A smartphone running the SGEMM benchmark.

We assume that items #1-3 are plugged into an already-existent wired network, while items #4-5 achieve network connectivity via the tree topology described in [Section 4.2](#)

Manufacturing (C_M). The inclusion of hardware peripherals adds a new term to C_M : $C_{M(p)}$, which represents the carbon cost of the added peripherals. The cloudlet-scale C_M for the repurposed smartphone- and laptop-based systems is:

$$C_M = N * C_{M(batt.)} * \left[\frac{lifetime}{battery\ lifetime} \right] + C_{M(p)} \quad (12)$$

where N is the number of devices.

Networking (C_N). We calculate the carbon footprint of networking as described in [Equation \(5\)](#):

$$C_N = \sum_{lifetime} C_{Igrid} * f_{net} * E_{Inet}$$

For the smartphones, E_{Inet} varies for 3G, 4G, and WiFi. From [1], we use 5 uJ/byte for WiFi, and 11 uJ/byte for LTE. The actual value of f_{net} will vary widely depending on the target workload. We assume 0.1 Gbps for each cloudlet.

Compute (C_C). At cloudlet scale, we add the operational carbon cost of any added peripherals. For the smartphone-based cloudlets, this will be the added power consumption of the server fans.

$$C_C = N * C_{C(device)} + C_{C(p)} \quad (13)$$

The results of this cloudlet-scale calculation are presented in [Figure 5](#). We plot the results across three benchmarks: SGEMM, PDF Rendering, and Dijkstra, and for two power regimes, California mix and 100% solar. In the 100% solar regime, we remove the smartplugs from the equation since they are not needed for smart charging.

In all cases, the repurposed smartphones and laptops perform better than the new server, especially for shorter lifetimes. The old server performs the worst overall due to its relatively high energy consumption ($2/3$ that of the PowerEdge despite being significantly less powerful). For the SGEMM benchmark, the ThinkPad performs the best overall; for the other two benchmarks, the Pixel performs the best. The carbon savings become even more pronounced in the 100% solar regime, where embodied carbon dominates.

It is interesting to note that the Nexus 4 smartphone cluster, despite consuming more energy (456 W) than the new PowerEdge

Table 4: Theoretical three-year datacenter-scale CCI projections for a PowerEdge and smartphone-based system.

	SGEMM	PDF Render	Dijkstra
	mgCO ₂ -e/gflop	mgCO ₂ -e/M/pixel	mgCO ₂ -e/MTE
PowerEdge	0.598	0.394	15.4
Smartphone	0.292	0.021	0.691

(309 W), is nonetheless still more carbon efficient for both the PDF Rendering and Dijkstra benchmarks. For the SGEMM benchmark, the Nexus 4 cluster is more carbon efficient for lifetimes less than 45 months. In other words, running the higher-powered Nexus 4 phone cluster is better than manufacturing a new server if that server will be in service for less than 45 months. The more efficient Pixel 3A smartphone cluster beats out the server every time.

5.3 Datacenter-Scale Analysis

Like PUE, CCI can be calculated at the datacenter scale and provides new insights not reflected by PUE alone.

To illustrate this, we estimate CCI and PUE for two 50MW datacenters: One built with clusters of Pixel 3A smartphones and one with PowerEdge R740 servers. With the PowerEdge at 308 W and the Pixel 3A cluster at 84 W (54 Pixel 3A phones on a light-medium workload), we provision each 50 MW datacenter with 170,000 units. We assume each smartphone cluster will take up 2U of space, which is enough to leave 75% of the space empty.

5.3.1 PUE. We make the simplifying assumptions of no personnel or windows and define PUE as:

$$PUE = \frac{P_{IT} + P_{cooling} + P_{lighting}}{P_{IT}} \quad (14)$$

We follow the methodology in [42] to get estimates for $P_{cooling}$ and $P_{lighting}$, using the wattage and size of the PowerEdge and phone cluster. This gives a PUE of 1.32 for the smartphone-based design and 1.31 for the traditional server-based system. The smartphone-based design has a slightly larger PUE because it takes up more physical space, which means that $P_{cooling}$ and $P_{lighting}$ are higher.

5.3.2 CCI. For datacenter scale:

$$CCI = \frac{C_M + PUE * (C_C + C_N)}{\sum_{lifetime} flop} \quad (15)$$

Using the PUE results from above, and assuming a California energy mix and three-year lifespan, we get the macro-level CCI results given in [Table 4](#).

6 JUNKYARD CLOUDLET

To demonstrate the feasibility of smartphone-based cloudlet computing we build out a proof-of-concept cluster of ten Pixel 3A smartphones and evaluate our junkyard prototype against a range of similarly-powerful AWS EC2 instances using the DeathStar-Bench benchmarking suite [19], which provides complete end-to-end microservice-based cloud applications.

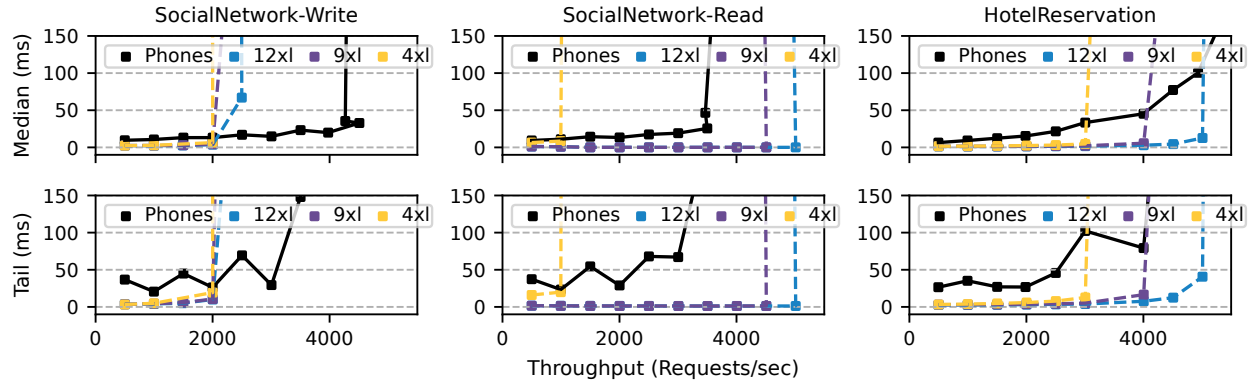


Figure 7: The performance of the smartphone cloudlet vs. different AWS EC2 instances for the three DeathStarBench workloads. The top graph shows the median (50%) latency and bottom graph gives the tail (90%) latency. For the SocialNetwork-Write, the smartphone cluster outperforms a c5.12xlarge instance; however, for the SocialNetwork-Read, its performance is between c5.4xlarge and c5.9xlarge. For the HotelReservation with its mixed workload, the performance is similar to a c5.9xlarge instance.

6.1 Experimental Methodology

The testbed consists of ten Google Pixel 3A and Pixel 3A XL phones, initially released in 2019. We replace the native Android OS with Ubuntu Touch, an open-source mobile OS that provides a desktop-like experience [51]. To support Docker, which DeathStarBench relies upon, we further modify the Ubuntu Touch kernel to add several necessary modules, e.g., the BTRFS file system [34].

Benchmarks. We implemented two of the three publicly available end-to-end applications from the DeathStarBench suite. They are described below based on information from the DeathStarBench paper and public GitHub [14, 19].

- *HotelReservation* is a service that supports getting location-based hotel information and rates, recommending hotels based on user metrics, and placing reservations. It is built with Go and gRPC. The benchmark includes a mixed workload generator.
- *SocialNetwork* implements unidirectional follow relationships and supports creating text and media posts, reading posts, reading an entire user timeline, searching a database for users or posts, registering, logging in, and following. It is implemented in C++ and uses Thrift RPCs for communication. There are three workload generators: composing posts, reading user timelines, and reading home timelines. We present results for the first two workload generators.

We attempted to deploy the third public DeathStarBench application (MovieReviewing) on the smartphone cloudlet but found that it did not scale well to multiple devices. The median and tail latency increased with an increasing number of devices. This has been observed by others [30], so we do not believe that it is a limitation of the smartphone platform but rather a property of the benchmark.

AWS Experiments. As a baseline, we test each benchmark on differently-sized AWS EC2 C5 instances, described as providing “cost-effective high performance” for “advanced compute-intensive workloads” [49]. The applications run on a single AWS machine. The client workload generator operates in another thread on the

same machine to eliminate network latency between client and server. Since all microservices are deployed on the same node, there is also no network latency between microservice function calls.

Smartphone Testbed. The smartphones are connected over WiFi. They run in Docker Swarm mode, which distributes microservices amongst the phones according to the dependencies defined in the `docker-compose-swarm.yml` file from the DeathStarBench GitHub [14]. To minimize network latency, all phones and the client are on the same local WiFi. Device communication has added latency since the microservices are spread amongst multiple devices.

6.2 DeathStarBench Performance

Figure 7 shows the performance of the phone cloudlet against various AWS EC2 instances. The smartphone cloudlet has a higher tail and median latency across all throughputs; however, it scales well to a large number of requests. This higher latency is expected, given the added network latency between devices in the swarm.

The latency requirements of the system would determine whether the smartphone cloudlet is an acceptable replacement for a cloud server. For instance, if the hotelReservation system required that median latency be no more than 50 ms and tail latency no more than 100 ms, the smartphone cloudlet could meet the specs and handle up to 4,000 queries per second. In this case, the performance of the cloudlet would approximate that of a c5.9xlarge EC2 instance, which costs \$1.53 per hour.

The smartphone cloudlet costs \$1,027.60 USD for three years of deployment, which includes the initial \$70 USD cost per Pixel 3A and California energy prices. The c5.9xlarge EC2 instance costs \$40,404 USD (\$1.53 USD per hour) for the same deployment length.

Some interesting differences exist between the smartphone cluster performance on the different benchmarks. Normalized to the AWS equivalents, the smartphone cluster performs significantly better on the write-only SocialNetwork application than on the read-only application. This may be explained by the fact that the read workload involves the transfer of a user’s entire timeline – a large amount of data.

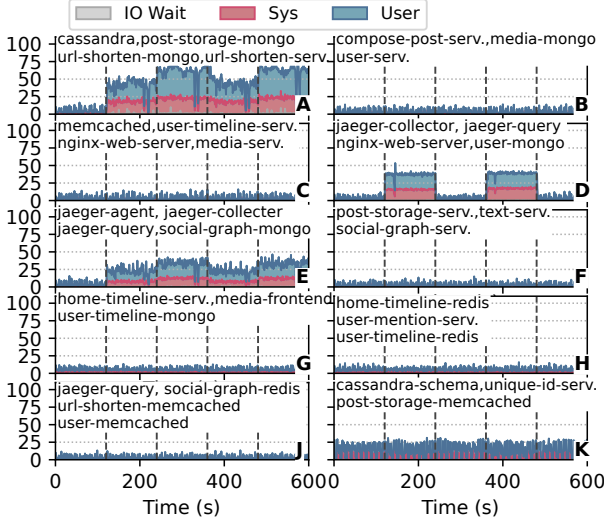


Figure 8: CPU utilization for each of the ten smartphones running the SocialNetwork benchmark. The system is idle from 0-120s. The SocialNetwork-Read workload is run at 3,000 QPS from 120-240s. No requests are submitted from 240-360s. The SocialNetwork-Write workload is run at 3,500 QPS from 360-480s. The system is idle again from 480-600s. The vertical dotted lines indicate the starting/stopping points. The microservices hosted by each device are shown on the graph.

CPU Utilization. The CPU utilization of the smartphones (Figure 8) indicates two interesting points. First, the smartphones were generally not CPU-bound while hosting these microservices. Second, the utilization varied widely depending on the services being hosted by the device, with 6/10 of the devices making little use of their raw compute powers.

We ran the same experiment on the c5.9xlarge and found that it was similarly not CPU-bound: the CPU utilization was relatively constant at 30% utilization while running the SocialNetwork-Read workload at 4,500 QPS and 25% while running the SocialNetwork-Write workload at 2,000 QPS.

6.3 Carbon Performance

A Pixel 3A consumed approximately 1.7 W while hosting the hotel reservation system. Therefore, we assume that the ten-device system running at 17 W requires, at most, a single server fan. With this energy consumption, the phones would have to be charged 3.7 times a day, and batteries would have to be replaced every 1.9 years. Using the CCI methodology, Figure 9 shows the carbon intensity of the Pixel 3A cloudlet.

It is impossible to know the actual power consumption of the EC2 instances without having access to their custom hardware. We use the public dataset of rough estimates based on benchmarking very similar hardware [13] to approximate that a c5.9xlarge instance will use 140.7 W at 10% utilization, and 239 W at 50%. The CPU benchmarking indicates that the c5.9xlarge will likely be at 25-30% utilization while hosting SocialNetwork; for simplicity, we round down to 10% and use the 140.7 W as an estimate for c5.9xlarge. The

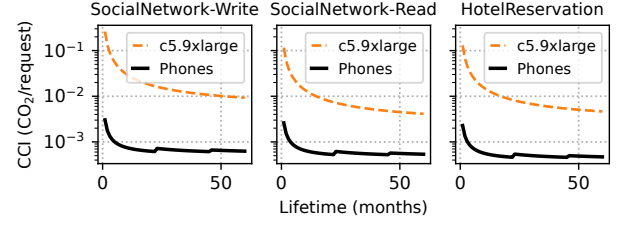


Figure 9: Carbon intensity of Pixel 3A cloudlet running the DeathStarBench applications compared to c5.9xlarge server.

same dataset estimates the manufacturing carbon of the c5.9xlarge as 1344 kg CO₂-equivalent.

The throughput of each alternative is determined by considering the point on the curves in Figure 7 at which throughput is at its max before the latencies shoot up. For the smartphone cluster, this is 4,000 QPS for HotelReservation, 3,000 QPS for SocialNetwork-Write, and 3,500 QPS for SocialNetwork-Read. The throughput for C5.9xlarge is 4,000 QPS for HotelReservation, 4,500 QPS for SocialNetwork-Read, and 2,000 QPS for SocialNetwork-Write.

Figure 9 shows the final result. For each benchmark studied, the smartphone-based system is significantly more carbon efficient, in terms of CO₂ per query. After three years of use, the relative carbon efficiency of the smartphone cluster vs. the c5.9xlarge singlet is 18.9x more carbon efficient for SocialNetwork-Write, 9.8x more carbon efficient for SocialNetwork-Read, and 12.6x more carbon efficient for HotelReservation with its mixed workload.

7 CONCLUSION

The major takeaways of this work are:

- (1) For specific workloads, clusters of repurposed phones are cheaper and more carbon efficient than traditional servers.
- (2) More broadly, scavenging unwanted equipment shows excellent potential for building economic and carbon-efficient systems, especially when renewable energy is plentiful.
- (3) Sustainability has operational and manufacturing facets; manufacturing dominates as operating trends towards zero with cleaner energy mixes.
- (4) Accurate LCA information is essential for carbon-based analyses; it would be beneficial if more ICT manufacturers published this information, including cloud providers who build custom systems.

Our work highlights the need for more holistic analyses of the environmental impact of computing. With the substantial carbon cost of manufacturing and the difficulties of responsible recycling, the energy efficiency of a device may be the least significant component of its environmental and human impact.

8 ACKNOWLEDGEMENTS

This work was funded in part by a National Science Foundation (NSF) grant (CNS-2233894). We thank the anonymous reviewers and our shepherd for their aid in improving the paper. We would also like to thank Dean Tullsen for his feedback and encouragement.

REFERENCES

- [1] Ramón Agüero, Thomas Zinner, Mario García-Lozano, Bernd-Ludwig Wenning, and Andreas Timm-Giel. 2016. *Mobile Networks and Management: 7th International Conference, MONAMI 2015, Santander, Spain, September 16–18, 2015, Revised Selected Papers*. Vol. 158. Springer.
- [2] Amazon. 2021. Amazon EC2 T4g Instances. <https://aws.amazon.com/ec2/instance-types/t4/>. Accessed: 2021-08-01.
- [3] David G Andersen, Jason Franklin, Michael Kaminsky, Amar Phanishayee, Lawrence Tan, and Vijay Vasudevan. 2009. FAWN: A fast array of wimpy nodes. In *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*. 1–14.
- [4] Apple. 2020. 13-inch MacBook Pro PER. https://www.apple.com/environment/pdf/products/notebooks/13-inch_MacBookPro_PER_Nov2020.pdf. Accessed: 2022-02-28.
- [5] Apple. 2021. Product Environmental Report iPad (9th generation). https://www.apple.com/environment/pdf/products/ipad/iPad_PER_Sept2021.pdf. Accessed: 2022-02-28.
- [6] Johannes Baeckman. [n. d.]. Mobile Phone Waste. In *CONFERENCE IN INTERACTION TECHNOLOGY AND DESIGN*. 115.
- [7] Cornelis P Baldé, Vanessa Forti, Vanessa Gray, Ruediger Kuehr, and Paul Stegmann. 2017. *The global E-waste monitor 2017: Quantities, flows and resources*. United Nations University, International Telecommunication Union, and
- [8] M. Brannon, P. Graeter, D. Schwartz, and J. R. Santos. 2014. Reducing electronic waste through the development of an adaptable mobile device. In *2014 Systems and Information Engineering Design Symposium (SIEDS)*. 57–62. <https://doi.org/10.1109/SIEDS.2014.6829871>
- [9] Knut Breivik, James M Armitage, Frank Wania, and Kevin C Jones. 2014. Tracking the global generation and exports of E-waste. Do existing estimates add up? *Environmental science & technology* 48, 15 (2014), 8735–8743.
- [10] Andreas Busa. 2019. Life Cycle Assessment of Dell R740 Server. https://www.delltechnologies.com/asset/en-us/products/servers/technical-support/Full_LCA_Dell_R740.pdf. Accessed: 2021-06-01.
- [11] Felix Büsching, Sebastian Schildt, and Lars Wolf. 2012. Droidcluster: Towards smartphone cluster computing—the streets are paved with potential computer clusters. In *2012 32nd International Conference on Distributed Computing Systems Workshops*. IEEE, 114–117.
- [12] Wikipedia Contributors. 2021. IEEE 802.11n-2009. https://en.wikipedia.org/wiki/IEEE_802.11n-2009. Accessed: 2021-11-23.
- [13] Benjamin Davy. 2021. Building an AWS EC2 Carbon Emissions Dataset. <https://medium.com/teads-engineering/building-an-aws-ec2-carbon-emissions-dataset-3f0fd76c98ac>. Accessed: 2022-10-17.
- [14] Christina Delimitrou. 2022. DeathStarBench. <https://github.com/delimitrou/DeathStarBench/>. Accessed: 2022-08-01.
- [15] Dell. 2019. PowerEdge R640 Estimated Product Carbon Footprint. https://i.dell.com/sites/csdocuments/CorpComm_Docs/en/carbon-footprint-poweredge-r640.pdf
- [16] Enermax. 2022. LIQTECH TR4 II series 280mm CPU liquid cooler. <https://www.enermax.com/en/products/liqtech-tr4-ii-series-280mm-cpu-liquid-cooler>. Accessed: 2022-02-17.
- [17] Mine Ercan, Jens Malmmodin, Pernilla Bergmark, Emma Kimfalk, and Ellinor Nilsson. 2016. Life cycle assessment of a smartphone. *Proceedings of the ICT for Sustainability, Amsterdam, The Netherlands* (2016), 29–31.
- [18] Power Bank Expert. 2021. How Long Do Cell Phone Batteries Last? <https://www.powerbankexpert.com/how-long-do-cell-phone-batteries-last/>. Accessed: 2021-08-13.
- [19] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyari Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, Kelvin Hu, Meghna Pancholi, Yuan He, Brett Clancy, Chris Colen, Fukang Wen, Catherine Leung, Siyuan Wang, Leon Zaruvinsky, Mateo Espinosa, Rick Lin, Zhongling Liu, Jake Padilla, and Christina Delimitrou. 2019. An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (Providence, RI, USA) (ASPLoS '19). Association for Computing Machinery, New York, NY, USA, 3–18. <https://doi.org/10.1145/3297858.3304013>
- [20] GeekBench. 2021. <https://browser.geekbench.com/v4/cpu/search>. Accessed: 2021-11-23.
- [21] Geekbench. 2021. Android Benchmarks - Geekbench. <https://browser.geekbench.com/android-benchmarks>. Accessed: 2021-08-01.
- [22] Google. [n. d.]. 24/7 Carbon-Free Energy by 2030. <https://www.google.com/about/datacenters/cleanenergy/>. Accessed: 2022-02-28.
- [23] Boris Grot, Damien Hardy, Pejman Lotfi-Kamran, Babak Falsafi, Chrysostomos Nicopoulos, and Yiannakis Sazeides. 2012. Optimizing Data-Center TCO with Scale-Out Processors. *IEEE Micro* 32, 5 (2012), 52–63. <https://doi.org/10.1109/MM.2012.71>
- [24] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: designing sustainable computer systems with an architectural carbon modeling tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 784–799.
- [25] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2021. Chasing Carbon: The Elusive Environmental Footprint of Computing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 854–867.
- [26] Ali Habibi Khalaj and Saman K. Halgamuge. 2017. A Review on efficient thermal management of air- and liquid-cooled data centers: From chip to the cooling system. *Applied Energy* 205 (2017), 1165–1188. <https://doi.org/10.1016/j.apenergy.2017.08.037>
- [27] Stavros Harizopoulos and Spiros Papadimitriou. 2011. A Case for Micro-Cellstores: Energy-Efficient Data Management on Recycled Smartphones. In *Proceedings of the Seventh International Workshop on Data Management on New Hardware* (Athens, Greece) (DaMoN '11). Association for Computing Machinery, New York, NY, USA, 50–55. <https://doi.org/10.1145/1995441.1995448>
- [28] I.M.S.K. Ilankoon, Yousef Ghorbani, Meng Nan Chong, Gamini Herath, Thandazile Moyo, and Jochen Petersen. 2018. E-waste in the international context – A review of trade flows, regulations, hazards, waste management strategies and technologies for value recovery. *Waste Management* 82 (2018), 258–275. <https://doi.org/10.1016/j.wasman.2018.10.018>
- [29] California ISO. 2021. Supply and renewables. <http://www.caiso.com/todaysoutlook/pages/supply.aspx>. Accessed: 2021-11-18.
- [30] Zhipeng Jia and Emmett Witchel. 2021. Nightcore: Efficient and Scalable Serverless Computing for Latency-Sensitive, Interactive Microservices. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Virtual, USA) (ASPLoS '21). Association for Computing Machinery, New York, NY, USA, 152–166. <https://doi.org/10.1145/3445814.3446701>
- [31] Aditya Joshi, Abhishek Gupta, Shalini Verma, Akshoy Ranjan Paul, Anuj Jain, and Nawshad Haque. 2021. Life Cycle Based Greenhouse Gas Footprint Assessment of a Smartphone. In *IOP Conference Series: Earth and Environmental Science*, Vol. 795. IOP Publishing, 012028.
- [32] Soowon Kang, Hyeonwoo Choi, Sooyoung Park, Chunjong Park, Jemin Lee, Uichin Lee, and Sung-Ju Lee. 2019. Fire in your hands: Understanding thermal behavior of smartphones. In *The 25th Annual International Conference on Mobile Computing and Networking*. 1–16.
- [33] Daniel Keeble. 2013. The Culture of Planned Obsolescence in Technology Companies. Bachelor's Thesis.
- [34] The kernel development community. [n. d.]. BTRFS. <https://docs.kernel.org/filesystems/btrfs.html>.
- [35] Kyung Mo Kim, Yeong Shin Jeong, and In Cheol Bang. 2019. Thermal analysis of lithium ion battery-equipped smartphone explosions. *Engineering Science and Technology, an International Journal* 22, 2 (2019), 610–617.
- [36] Noah Klugman, Meghan Clark, Pat Pannuto, and Prabal Dutta. 2018. Android Resists Liberation from Its Primary Use Case. In *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking* (New Delhi, India) (MobiCom '18). Association for Computing Machinery, New York, NY, USA, 849–851. <https://doi.org/10.1145/3241539.3267726>
- [37] Amit Kumar, Maria Holuszko, and Denise Croce Romano Espinosa. 2017. E-waste: An overview on generation, collection, legislation and recycling practices. *Resources, Conservation and Recycling* 122 (2017), 32–42. <https://doi.org/10.1016/j.resconrec.2017.01.018>
- [38] Bo Li, Jianxin Yang, Xiaolong Song, and Bin Lu. 2012. Survey on disposal behaviour and awareness of mobile phones in Chinese university students. *Procedia Environmental Sciences* 16 (2012), 469–476.
- [39] Hui Li, Liping Yu, Guoying Sheng, Jiamo Fu, and Ping'an Peng. 2007. Severe PCDD/F and PBDD/F pollution in air around an electronic waste dismantling area in China. *Environmental Science & Technology* 41, 16 (2007), 5641–5646.
- [40] Tom Murphy. [n. d.]. Space-Based Solar Power. <https://dothemath.ucsd.edu/2012/03/space-based-solar-power/>. Accessed: 2022-02-17.
- [41] Byunggook Na, Jaehye Jang, Seongsik Park, Seijoon Kim, Joonoo Kim, Moon Sik Jeong, Kwang Choon Kim, Seon Heo, Yoonsang Kim, and Sungroh Yoon. 2021. Scalable Smartphone Cluster for Deep Learning. *arXiv preprint arXiv:2110.12172* (2021).
- [42] netcom. 2022. Server Room Air Cooling Calculation Guide. <https://www.netcomtech.co.uk/airconcalculation/>. Accessed: 2022-03-01.
- [43] Zhonghong Ou, Bo Pang, Yang Deng, Jukka K. Nurminen, Antti Ylä-Jääski, and Pan Hui. 2012. Energy- and Cost-Efficiency Analysis of ARM-Based Clusters. In *2012 12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (ccgrid 2012)*. 115–123. <https://doi.org/10.1109/CCGrid.2012.84>
- [44] Daniel Pargman and Barath Raghavan. 2014. Rethinking sustainability in computing: From buzzword to non-negotiable limits. In *Proceedings of the 8th Nordic Conference on Human-Computer Interaction: Fun, Fast, Foundational*. 638–647.
- [45] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350* (2021).
- [46] Barath Raghavan and Justin Ma. 2011. The energy and energy of the internet. In *Proceedings of the 10th ACM Workshop on hot topics in networks*. 1–6.

- [47] Nikola Rajovic, Paul M Carpenter, Isaac Gelado, Nikola Puzovic, Alex Ramirez, and Mateo Valero. 2013. Supercomputing with commodity CPUs: Are mobile SoCs ready for HPC?. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. 1–12.
- [48] Nikola Rajovic, Alejandro Rico, Nikola Puzovic, Chris Adeniyi-Jones, and Alex Ramirez. 2014. Tibidabo: Making the case for an ARM-based HPC system. *Future Generation Computer Systems* 36 (2014), 322–334.
- [49] Amazon Web Services. [n.d.]. Amazon EC2 C5 Instances. [AmazonEC2C5Instances](#). Accessed: 2022-10-30.
- [50] Mohammad Shahradd and David Wentzlaff. 2017. Towards deploying decommissioned mobile devices as cheap energy-efficient compute nodes. In *9th {USENIX} Workshop on Hot Topics in Cloud Computing (HotCloud 17)*.
- [51] ubports. 2021. Ubuntu Touch. <https://ubuntu-touch.io/>. Accessed: 2022-04-12.
- [52] Emile Van Eygen, Steven De Meester, Ha Phuong Tran, and Jo Dewulf. 2016. Resource savings by urban mining: The case of desktop and laptop computers in Belgium. *Resources, Conservation and Recycling* 107 (2016), 53–64. <https://doi.org/10.1016/j.resconrec.2015.10.032>
- [53] Trevor Zink, Frank Maker, Roland Geyer, Rajeevan Amirtharajah, and Venkatesh Akella. 2014. Comparative life cycle assessment of smartphone reuse: repurposing vs. refurbishment. *The International Journal of Life Cycle Assessment* 19, 5 (2014), 1099–1109.

Received 2022-07-07; accepted 2022-09-22